AG TECHNOLOGIES

Architecture de Services Partagés

Cahier des Charges Technique

Service d'Authentification Centralisé

Auth Service — AGT-Auth

Version	2.1 (final — aligned with Users v2.1)
Date	Mars 2026
Statut	Implementation-ready
Classification	Confidentiel
Auteur	Équipe Architecture

Document confidentiel — Réservé à l'usage interne d'AG Technologies

Historique des Révisions

Ver.	Date	Auteur	Description
1.0	2026-03-19	Équipe Archi.	Création initiale
1.1	2026-03-20	Équipe Archi.	Post-audit : stack unifiée Django+DRF+Django ORM ; rôles supprimés (délégués au Service Users) ; endpoints ajoutés (plateformes CRUD, verify-otp, magic-link callback, admin block/unblock, health) ; MLD complété (verification_tokens, registration_platform_id, is_deactivated) ; clarification verify-token vs introspect ; JWT documenté
1.2	2026-03-20	Équipe Archi.	Bloquants résolus : modèle admin (API key statique) ; règles register (phone sans mdp, un identifiant à la fois, OAuth flux séparé) ; verify-token vérification complète (blocked, deactivated, platform, session) ; règles de révocation sessions formalisées ; callbacks OAuth/magic-link sécurisés (cookie HttpOnly) ; max 5 refresh tokens par user
1.3	2026-03-20	Équipe Archi.	Production-ready : client_secret hashé (bcrypt) ; contrainte unique (provider, provider_user_id) sur oauth_providers ; session_id ajouté au JWT payload ; refresh token via cookie HttpOnly uniquement ; SameSite=Lax ; corrections cohérence documentaire
1.3.1	2026-03-20	Équipe Archi.	Alignement final : refresh_token retiré de toutes les réponses JSON (cookie-only) ; effet du logout sur l'access token précisé ; mini-contrat Auth → Notification ajouté
1.3.2	2026-03-20	Équipe Archi.	Endpoint POST /auth/token/exchange ajouté (cookie → bearer après OAuth/magic link) ; stratégie CSRF documentée ; coquilles de version corrigées
2.0	2026-03-20	Équipe Archi.	Alignement Users v2.1 : endpoints admin/deactivate et admin/purge ajoutés ; contrat Auth → Users (status-sync) formalisé ; besoins F-USR-03/F-USR-04 ajoutés ; règles révocation complétées ; mapping statuts Auth ↔ Users ; clarification identité userId ; terminologie block/deactivate/purge harmonisée
2.1	2026-03-20	Équipe Archi.	Version finale : contrat sync email/phone ajouté ; propagation status-sync sur account/deactivate ; comportement transactionnel admin/purge ; réactivation clarifiée (aucun flux en v2.1) ; statut document passé à Implementation-ready

Contents

1 Introduction **5**
1.1 Périmètre . . . 5
1.2 Hors périmètre . . . 5

2 Contexte & Problématique **5**
2.1 Contexte . . . 5
2.2 Problématique . . . 5

3 Vision & Objectifs **5**
3.1 Vision . . . 5
3.2 Objectifs . . . 6

4 Parties Prenantes & Utilisateurs **6**

5 Besoins Fonctionnels **6**
5.1 Inscription (Register) . . . 6
5.2 Connexion (Login) . . . 6
5.3 Gestion de session & tokens . . . 7
5.4 Sécurité & 2FA . . . 7
5.5 Traçabilité & Audit . . . 7
5.6 Gestion des Plateformes (Admin) . . . 7
5.7 Gestion du Compte Utilisateur . . . 8

6 Besoins Non-Fonctionnels **8**

7 Architecture Technique **8**
7.1 Vue d'ensemble . . . 8
7.2 Stack Technique . . . 9
7.3 Dépendances Externes & Règles d'Architecture . . . 9
7.4 Format du JWT Payload . . . 9

8 Modélisation **10**
8.1 Diagramme de Classes . . . 10
8.2 MLD — Modèle Logique de Données . . . 10
8.2.1 Table `platforms` . . . 10
8.2.2 Table `users_auth` . . . 11
8.2.3 Table `sessions` . . . 11
8.2.4 Table `refresh_tokens` . . . 12
8.2.5 Table `oauth_providers` . . . 12
8.2.6 Table `login_history` . . . 12
8.2.7 Table `verification_tokens` . . . 12

9 Contrats d'API **13**
9.1 Health Check . . . 13
9.1.1 GET /auth/health . . . 13
9.2 Inscription & Vérification . . . 13
9.2.1 POST /auth/register . . . 13
9.2.2 POST /auth/verify-email . . . 14
9.2.3 POST /auth/verify-otp . . . 14
9.3 Authentification (Login) . . . 15
9.3.1 POST /auth/login . . . 15

9.3.2 POST /auth/login/phone . . . 15
9.3.3 POST /auth/login/magic-link . . . 16
9.3.4 GET /auth/magic-link/callback . . . 16
9.4 Authentification Sociale (OAuth) . . . 16
9.4.1 GET /auth/oauth/google . . . 16
9.4.2 GET /auth/oauth/google/callback . . . 17
9.4.3 GET /auth/oauth/facebook . . . 17
9.4.4 GET /auth/oauth/facebook/callback . . . 17
9.5 Sécurité & Mots de passe . . . 17
9.5.1 POST /auth/forgot-password . . . 17
9.5.2 POST /auth/reset-password . . . 17
9.5.3 PUT /auth/change-password . . . 18
9.6 Authentification à Double Facteur (2FA) . . . 18
9.6.1 POST /auth/2fa/enable . . . 18
9.6.2 POST /auth/2fa/confirm . . . 18
9.6.3 POST /auth/2fa/verify . . . 19
9.6.4 POST /auth/2fa/disable . . . 19
9.7 Gestion des Sessions & Tokens . . . 19
9.7.1 POST /auth/refresh . . . 19
9.7.2 POST /auth/logout . . . 20
9.7.3 GET /auth/sessions . . . 20
9.7.4 DELETE /auth/sessions/{sessionId} . . . 20
9.7.5 GET /auth/verify-token . . . 21
9.8 Règles de Révocation des Sessions . . . 21
9.9 Échange de Token (Cookie → Bearer) . . . 22
9.9.1 POST /auth/token/exchange . . . 22
9.10 Profil & Audit . . . 23
9.10.1 GET /auth/me . . . 23
9.10.2 GET /auth/login-history . . . 23
9.10.3 GET /auth/stats/{userId} . . . 24
9.11 Gestion des Plateformes (Admin) . . . 24
9.11.1 POST /auth/platforms . . . 24
9.11.2 GET /auth/platforms . . . 24
9.11.3 PUT /auth/platforms/{platformId} . . . 25
9.11.4 DELETE /auth/platforms/{platformId} . . . 25
9.12 Administration . . . 25
9.12.1 POST /auth/admin/block/{userId} . . . 25
9.12.2 POST /auth/admin/unblock/{userId} . . . 26
9.12.3 POST /auth/account/deactivate . . . 26
9.12.4 POST /auth/admin/deactivate/{authUserId} . . . 26
9.12.5 DELETE /auth/admin/purge/{authUserId} . . . 27
9.13 Tokens Inter-Services (Machine-to-Machine) . . . 27
9.13.1 POST /auth/s2s/token . . . 27
9.13.2 POST /auth/s2s/introspect . . . 28

10 Infrastructure **28**
10.1 Composants d'infrastructure . . . 28
10.2 Services externes à payer . . . 28
10.3 Variables d'Environnement . . . 29

11 Stratégie de Gestion des Tokens (v1.3) **29**

12 Rate Limiting **30**

13 Contrat d'Intégration Auth → Notification **30**
13.1 Mode de communication 30
13.2 Payload standard 30
13.3 Types de notifications émis par Auth 31
13.4 Comportement en cas d'échec 31

14 Contrat d'Intégration Auth → Users **31**
14.1 Provisioning 31
14.2 Synchronisation de statut 32
14.3 Synchronisation email / téléphone 32
14.4 Mapping de statuts Auth ↔ Users 32

15 Recommandations & Évolutions **33**

1 Introduction

Ce document constitue le cahier des charges du **Service d'Authentification Centralisé** (ci-après « Auth Service ») destiné à l'écosystème AGT. Il sert de référence unique pour la conception, le développement et les décisions techniques.

1.1 Périmètre

Le service Auth couvre le cycle de vie de l'**identité pure** : inscription, connexion, gestion de sessions, sécurité (2FA, rate limiting), administration des plateformes clientes et traçabilité.

1.2 Hors périmètre

- Gestion du profil étendu (déléguée au Service Users)
- Gestion des rôles et permissions (déléguée au Service Users)
- Logique métier propre aux plateformes consommatrices
- Envoi effectif d'emails/SMS (délégué au Service Notification)
- Le cycle de vie du profil utilisateur, les états `deleted` et `deletion_in_progress`, ainsi que la logique de sortie d'une plateforme, sont gérés par le Service Users

2 Contexte & Problématique

2.1 Contexte

Plateforme	Statut	Description
AGT-Bot	En développement	Plateforme de bots automatisés
AGT-Market	En développement	Marketplace de services
SALMA	Migration prévue	Plateforme existante à intégrer
Futures	Planifié	Nouvelles plateformes à venir

2.2 Problématique

Sans service centralisé, chaque plateforme implémente sa propre authentification, entraînant : duplication du code de sécurité, expérience utilisateur fragmentée, maintenance coûteuse et risques de failles incohérentes.

3 Vision & Objectifs

3.1 Vision

Un utilisateur s'authentifie une seule fois et accède à toutes les plateformes de l'écosystème. Les rôles et permissions sont gérés par le Service Users.

3.2 Objectifs

#	Objectif	Indicateur
O1	Authentification unique	Un seul compte par utilisateur, multi-plateforme
O2	Sécurité renforcée	2FA, rate limiting, audit log complet
O3	Multi-méthodes	Email, phone, OAuth, magic link
O4	Traçabilité	Chaque action d'auth enregistrée
O5	Scalabilité	+10M utilisateurs, $<$ 200ms latence
O6	Réutilisabilité	Intégration nouvelle plateforme $<$ 1 jour

4 Parties Prenantes & Utilisateurs

Rôle	Type	Responsabilité
Product Owner	Interne	Priorisation des fonctionnalités
Équipe Backend	Interne	Développement et maintenance
Équipe Frontend	Interne	Intégration des flux d'authentification
DevOps	Interne	Infrastructure, CI/CD, monitoring
Utilisateurs finaux	Externe	Inscription, connexion, gestion de compte
Administrateurs	Interne	Gestion des plateformes, audit, supervision

Scalabilité attendue : +10 millions d'utilisateurs inscrits à travers l'écosystème.

5 Besoins Fonctionnels

5.1 Inscription (Register)

ID	Fonctionnalité	Description	Priorité
F-REG-01	Inscription email	Création de compte avec email + mot de passe	**Urgent**
F-REG-02	Inscription téléphone	Création de compte avec numéro + OTP SMS	Normale
F-REG-03	Inscription OAuth	Création via Google ou Facebook	**Urgent**
F-REG-04	Vérification email	Envoi d'un lien de vérification post-inscription	**Urgent**
F-REG-05	Vérification téléphone	Envoi OTP pour validation du numéro	Normale
F-REG-06	Liaison plateforme	Association automatique à la plateforme source	**Urgent**

5.2 Connexion (Login)

ID	Fonctionnalité	Description	Priorité
F-LOG-01	Login email/password	Authentification classique	**Urgent**
F-LOG-02	Login phone/OTP	Connexion via OTP SMS	Normale
F-LOG-03	Login OAuth	Connexion via Google / Facebook	**Urgent**
F-LOG-04	Magic link	Envoi d'un lien de connexion par email	**Urgent**
F-LOG-05	2FA challenge	Vérification du second facteur si activé	Normale
F-LOG-06	Émission tokens	JWT access token + refresh token	**Urgent**

5.3 Gestion de session & tokens

ID	Fonctionnalité	Description	Priorité
F-SES-01	Refresh token	Renouvellement du JWT via refresh token	**Urgent**
F-SES-02	Rotation refresh	Invalidation de l'ancien refresh token à chaque usage	**Urgent**
F-SES-03	Liste sessions	Visualisation des sessions actives	Normale
F-SES-04	Révocation session	Suppression d'une session spécifique	Normale
F-SES-05	Logout	Invalidation du refresh token et de la session	**Urgent**
F-SES-06	Verify token	Endpoint interne : validation d'un JWT utilisateur	**Urgent**

5.4 Sécurité & 2FA

ID	Fonctionnalité	Description	Priorité
F-SEC-01	Activation 2FA	Génération secret TOTP + QR code	Normale
F-SEC-02	Désactivation 2FA	Avec vérification du code actuel	Normale
F-SEC-03	Vérification 2FA	Validation code TOTP lors du login	Normale
F-SEC-04	Forgot password	Envoi d'un lien de réinitialisation	**Urgent**
F-SEC-05	Reset password	Réinitialisation via token temporaire	**Urgent**
F-SEC-06	Change password	Modification avec ancien mot de passe requis	**Urgent**

5.5 Traçabilité & Audit

ID	Fonctionnalité	Description	Priorité
F-AUD-01	Login history	Historique complet des connexions	**Urgent**
F-AUD-02	Méthode tracking	Enregistrement de la méthode utilisée	**Urgent**
F-AUD-03	Stats utilisateur	Fréquence par méthode et plateforme	Normale
F-AUD-04	Profil identité	Endpoint /auth/me (identité pure, sans rôles)	**Urgent**

5.6 Gestion des Plateformes (Admin)

ID	Fonctionnalité	Description	Priorité
F-PLAT-01	Création plateforme	Enregistrer une nouvelle plateforme cliente	**Urgent**
F-PLAT-02	Lister plateformes	Liste de toutes les plateformes et leur statut	**Urgent**
F-PLAT-03	Modifier plateforme	Mise à jour nom, méthodes d'auth autorisées	**Urgent**
F-PLAT-04	Supprimer plateforme	Désactivation d'une plateforme	**Urgent**

5.7 Gestion du Compte Utilisateur

ID	Fonctionnalité	Description	Priorité
F-USR-01	Désactivation de compte	Soft delete côté Auth (is_deactivated)	Normale
F-USR-02	Blocage admin	Blocage / déblocage d'un utilisateur	**Urgent**
F-USR-03	Purge RGPD	Suppression physique complète d'un compte Auth (appel inter-service)	**Urgent**
F-USR-04	Désactivation propagée	Désactivation S2S déclenchée par le Service Users (sans mot de passe)	**Urgent**

Note : la suppression définitive (hard delete / RGPD) est orchestrée par le Service Users, qui déclenche la purge dans Auth après avoir nettoyé le profil.

Terminologie (v2.0) :

- **Block** : mesure d'administration/sécurité. Temporaire ou réversible. Ne modifie pas le statut Users.
- **Deactivate** : fermeture logique du compte d'authentification. Propagée à Users comme `inactive`.
- **Purge** : suppression physique RGPD. Irréversible. Orchestrée par Users.

Identité (v2.0) :

Dans tous les endpoints `/auth/.../{userId}`, `userId` désigne `users_auth.id`, qui correspond à `auth_user_id` dans le Service Users.

6 Besoins Non-Fonctionnels

ID	Catégorie	Exigence	Cible
NF-01	Performance	Temps de réponse	$<$ 200ms (P95)
NF-02	Performance	Throughput	500 req/s min
NF-03	Disponibilité	Uptime	99.9%
NF-04	Sécurité	Hashing	bcrypt, cost $\geq$ 12
NF-05	Sécurité	JWT signature	RS256
NF-06	Sécurité	Rate limiting login	10 req/min
NF-07	Sécurité	Brute force	Blocage après 5 tentatives (15 min)
NF-08	Sécurité	Tokens	Access: 15 min, Refresh: 7 jours
NF-09	Scalabilité	Horizontal	Stateless, multi-instance
NF-10	Observabilité	Logs structurés	JSON, corrélation par request ID
NF-11	Observabilité	Health check	`GET /auth/health`

7 Architecture Technique

7.1 Vue d'ensemble

Le service Auth est un microservice REST stateless, déployé en conteneur Docker. Il expose une API JSON consommée par les plateformes clientes et les services internes.

7.2 Stack Technique

Composant	Technologie
Langage & Framework	Python 3.11+ / Django 5.x
API	Django REST Framework (DRF)
Base de données	PostgreSQL 15+
ORM	Django ORM
Migrations	Django Migrations
Cache / Sessions	Redis 7+
JWT	PyJWT (Signature RS256)
Hashing	bcrypt (cost factor $\geq$ 12)
2FA / TOTP	pyotp
Validation	DRF Serializers
Conteneurisation	Docker + Docker Compose
CI/CD	GitHub Actions

7.3 Dépendances Externes & Règles d'Architecture

Règle stricte de dépendance

Auth ne dépend d'**aucun service métier interne**, sauf le Notification Service (envoi emails/SMS).
Le User Service dépend de Auth (et non l'inverse). Auth gère l'identité pure et les credentials ; Users gère le profil, les rôles et les permissions.

Service	Type	Usage
Notification Service	Interne	Envoi emails (vérification, reset, magic link) et SMS (OTP)
Google OAuth	Externe	Authentification via compte Google
Facebook OAuth	Externe	Authentification via compte Facebook
SMS Provider	Externe	Twilio / Vonage / Infobip pour OTP

7.4 Format du JWT Payload

```
{
  "sub": "user-uuid",
  "iss": "agt-auth",
  "aud": "agt-ecosystem",
  "iat": 1710840000,
  "exp": 1710840900,
  "jti": "unique-token-id",
  "session_id": "session-uuid",
  "platform_id": "agt-bot",
  "email": "user@example.com",
  "email_verified": true,
  "two_fa_verified": false
}
```

Note v1.3 : `session_id` a été ajouté au payload pour permettre à `verify-token` de vérifier que la session est active. `jti` reste un identifiant unique du token lui-même (indépendant de la session).

Note : le JWT ne contient pas de rôles. Les services consommateurs doivent interroger le Service Users pour obtenir les rôles et permissions d'un utilisateur.

8 Modélisation

8.1 Diagramme de Classes

Le diagramme de classes complet est fourni en annexe au format PlantUML (voir section 15). Il couvre les entités : `Platform`, `UserAuth`, `Session`, `RefreshToken`, `OAuthProvider`, `LoginHistory` et `VerificationToken`.

8.2 MLD — Modèle Logique de Données

8.2.1 Table `platforms`

Colonne	Type	Contrainte	Description
id	UUID	PK	Identifiant unique
name	VARCHAR(50)	UNIQUE, NOT NULL	Nom (agt-bot, agt-market, salma)
slug	VARCHAR(50)	UNIQUE, NOT NULL	Slug URL-friendly
allowed_auth_methods	JSONB	NOT NULL	Méthodes activées
allowed_redirect_urls	JSONB	NOT NULL	Whitelist des URLs de redirection autorisées (OAuth, magic link)
client_secret_hash	VARCHAR(255)	NOT NULL	Hash bcrypt du secret S2S (le secret brut n'est affiché qu'à la création)
is_active	BOOLEAN	DEFAULT true	Plateforme active
created_at	TIMESTAMPTZ	NOT NULL	Création
updated_at	TIMESTAMPTZ	NOT NULL	Dernière mise à jour

8.2.2 Table users_auth

Colonne	Type	Contrainte	Description
id	UUID	PK	Identifiant unique
email	VARCHAR(255)	UNIQUE, NULL	Adresse email
phone	VARCHAR(20)	UNIQUE, NULL	Numéro de téléphone
password_hash	VARCHAR(255)	NULL	Hash bcrypt (cost 12+)
email_verified	BOOLEAN	DEFAULT false	Email vérifié
phone_verified	BOOLEAN	DEFAULT false	Téléphone vérifié
two_fa_enabled	BOOLEAN	DEFAULT false	2FA activé
two_fa_secret	VARCHAR(255)	NULL	Secret TOTP (chiffré AES)
registration_method	VARCHAR(20)	NOT NULL	email, phone, google, facebook
registration_platform_id	UUID	FK → platforms	Plateforme d'inscription initiale
failed_login_attempts	INT	DEFAULT 0	Compteur tentatives échouées
locked_until	TIMESTAMPTZ	NULL	Fin de blocage auto (brute force)
is_blocked	BOOLEAN	DEFAULT false	Blocage admin
is_deactivated	BOOLEAN	DEFAULT false	Soft delete (désactivation)
created_at	TIMESTAMPTZ	NOT NULL	Création
updated_at	TIMESTAMPTZ	NOT NULL	Dernière mise à jour

Ajouts v1.1 : `registration_platform_id` (F-REG-06), `is_blocked` (F-USR-02), `is_deactivated` (F-USR-01).

8.2.3 Table sessions

Colonne	Type	Contrainte	Description
id	UUID	PK	Identifiant session
user_id	UUID	FK → users_auth	Utilisateur
platform_id	UUID	FK → platforms	Plateforme de connexion
ip_address	VARCHAR(45)	NOT NULL	IP du client
user_agent	TEXT	NULL	User-Agent
is_active	BOOLEAN	DEFAULT true	Session active
created_at	TIMESTAMPTZ	NOT NULL	Début de session
expires_at	TIMESTAMPTZ	NOT NULL	Expiration

8.2.4 Table `refresh_tokens`

Colonne	Type	Contrainte	Description
id	UUID	PK	Identifiant
user_id	UUID	FK → users_auth	Utilisateur
session_id	UUID	FK → sessions	Session liée
token_hash	VARCHAR(255)	UNIQUE, NOT NULL	Hash du refresh token
is_revoked	BOOLEAN	DEFAULT false	Token révoqué
created_at	TIMESTAMPTZ	NOT NULL	Création
expires_at	TIMESTAMPTZ	NOT NULL	Expiration (7 jours)

8.2.5 Table `oauth_providers`

Colonne	Type	Contrainte	Description
id	UUID	PK	Identifiant
user_id	UUID	FK → users_auth	Utilisateur lié
provider	VARCHAR(20)	NOT NULL	google, facebook
provider_user_id	VARCHAR(255)	NOT NULL	ID chez le provider
email	VARCHAR(255)	NULL	Email du provider
created_at	TIMESTAMPTZ	NOT NULL	Création

Contraintes uniques : `(user_id, provider)` et `(provider, provider_user_id)`.

La seconde contrainte empêche qu'un même compte provider (ex : un Google ID) soit lié à plusieurs utilisateurs Auth.

8.2.6 Table `login_history`

Colonne	Type	Contrainte	Description
id	UUID	PK	Identifiant
user_id	UUID	FK → users_auth	Utilisateur
platform_id	UUID	FK → platforms	Plateforme
method	VARCHAR(20)	NOT NULL	email, phone, google, facebook, magic_link
ip_address	VARCHAR(45)	NOT NULL	IP
user_agent	TEXT	NULL	User-Agent
success	BOOLEAN	NOT NULL	Succès ou échec
failure_reason	VARCHAR(100)	NULL	Raison de l'échec
created_at	TIMESTAMPTZ	NOT NULL	Date/heure

8.2.7 Table `verification_tokens`

Ajout v1.1 : table pour stocker les tokens temporaires (vérification email, reset password, magic link, OTP). Les tokens sont également cachés dans Redis pour un lookup rapide.

Colonne	Type	Contrainte	Description
id	UUID	PK	Identifiant
user_id	UUID	FK → users_auth	Utilisateur concerné
token_hash	VARCHAR(255)	UNIQUE, NOT NULL	Hash SHA-256 du token
type	VARCHAR(30)	NOT NULL	email_verification, password_reset, magic_link, phone_otp
payload	JSONB	NULL	Données complémentaires (redirect_url, platform_id...)
expires_at	TIMESTAMPTZ	NOT NULL	Expiration
used_at	TIMESTAMPTZ	NULL	Date d'utilisation (NULL = non utilisé)
created_at	TIMESTAMPTZ	NOT NULL	Création

Règles :

- Un token ne peut être utilisé qu'une seule fois (`used_at` passe de NULL à la date).
- Les tokens expirés sont purgés par un cron job quotidien.
- Le token brut n'est jamais stocké ; seul le hash SHA-256 est persisté.
- Chaque token est également stocké dans Redis avec un TTL correspondant à son expiration pour un lookup O(1).

9 Contrats d'API

Base URL : `/api/v1`. Tous les endpoints retournent du JSON. Les endpoints protégés requièrent un header `Authorization: Bearer <token>`.

9.1 Health Check

9.1.1 GET /auth/health

Vérifie l'état du service (base de données, Redis). Aucune authentification requise.

Réponse 200 :

```
{
  "status": "healthy",
  "database": "ok",
  "redis": "ok",
  "version": "2.1.0"
}
```

9.2 Inscription & Vérification

9.2.1 POST /auth/register

Création d'un nouveau compte (email ou téléphone). **Le flux OAuth ne passe pas par cet endpoint** (voir section OAuth).
Headers : `X-Platform-Id: <platform_uuid>`
Request body :

```
{
  "email": "user@example.com",
  "password": "SecureP@ss123",
  "method": "email"
```

```
}
```

Règles de validation par méthode :

method	email	phone	password
`email`	Obligatoire	Ignoré	Obligatoire
`phone`	Ignoré	Obligatoire	Ignoré (auth par OTP uniquement)

- Un seul identifiant par inscription : email **ou** phone, jamais les deux.
- L'autre identifiant peut être ajouté ultérieurement (évolution future).
- Si `method=email`, le champ `phone` est ignoré (et inversement).
- Inscription OAuth : flux séparé via `GET /auth/oauth/{provider}`, création auto au callback.
- Inscription phone : pas de mot de passe en MVP, authentification exclusivement par OTP.

Response 201 :

```
{
  "id": "uuid",
  "email": "user@example.com",
  "email_verified": false,
  "registration_method": "email",
  "registration_platform_id": "platform-uuid",
  "message": "Verification email sent"
}
```

Codes : 201 Created, 400 Validation error, 409 Email/phone already exists.

9.2.2 POST /auth/verify-email

Request body :

```
{
  "token": "verification-token-from-email"
}
```

Response 200 :

```
{
  "message": "Email verified",
  "email_verified": true
}
```

9.2.3 POST /auth/verify-otp

Ajout v1.1 : endpoint générique pour vérifier un OTP, utilisé pour l'inscription téléphone ET le login phone.

Request body :

```
{
  "phone": "+237600000000",
  "otp_code": "482910",
  "context": "login",
  "platform_id": "uuid"
}
```

Le champ `context` peut valoir `"registration"` ou `"login"`.

Response 200 (context = login) :

```
{
  "access_token": "eyJhbG...",
  "token_type": "Bearer",
  "expires_in": 900
}
```

Le refresh token est posé dans un cookie HttpOnly (voir section Stratégie de Gestion des Tokens).

Response 200 (context = registration) :

```
{
  "message": "Phone verified",
  "phone_verified": true
}
```

Codes : 200 OK, 400 Invalid/expired OTP, 429 Rate limited.

9.3 Authentification (Login)

9.3.1 POST /auth/login

Connexion email + mot de passe.
Request body :

```
{
  "email": "user@example.com",
  "password": "SecureP@ss123",
  "platform_id": "uuid"
}
```

Response 200 :

```
{
  "access_token": "eyJhbG...",
  "token_type": "Bearer",
  "expires_in": 900,
  "requires_2fa": false
}
```

Le refresh token est posé dans un cookie HttpOnly.

Si `requires_2fa` est `true`, la réponse contient un `temp_token` au lieu de l'access token. Le client doit appeler `POST /auth/2fa/verify` pour finaliser.

Codes : 200 OK, 401 Invalid credentials, 403 Account locked/blocked/deactivated, 429 Rate limited.

9.3.2 POST /auth/login/phone

Demande d'OTP pour connexion par téléphone.
Request body :

```
{
  "phone": "+237600000000",
  "platform_id": "uuid"
}
```

Response 200 :

```
{
  "message": "OTP sent",
  "expires_in": 300
}
```

Le client soumet ensuite l'OTP reçu via `POST /auth/verify-otp` avec `context: "login"`.

9.3.3 POST /auth/login/magic-link

Envoi d'un lien de connexion par email.
Request body :

```
{
  "email": "user@example.com",
  "platform_id": "uuid",
  "redirect_url": "https://agt-bot.com/auth/callback"
}
```

Response 200 :

```
{
  "message": "Magic link sent",
  "expires_in": 600
}
```

9.3.4 GET /auth/magic-link/callback

Ajout v1.1, sécurisé v1.2 : endpoint de callback pour traiter le clic sur le magic link.

Query params : `?token=xxx`

Comportement (v1.2) : le callback ne retourne **pas** les tokens dans le body JSON ni dans l'URL. Il pose les tokens dans des **cookies HttpOnly sécurisés** (même mécanisme que les callbacks OAuth) puis redirige vers la `redirect_url` stockée dans le `payload` du `verification_token`.

Response : 302 Redirect vers `redirect_url` avec cookies posés :

- `Set-Cookie: access_token=eyJ...; HttpOnly; Secure; SameSite=Lax; Path=/; Max-Age=900`
- `Set-Cookie: refresh_token=dGh...; HttpOnly; Secure; SameSite=Lax; Path=/auth/refresh; Max-Age=604800`

Règles de sécurité :

- Token à usage unique : `used_at` est renseigné dès le premier clic.
- Token expiré ou déjà utilisé : 400 Bad Request.
- Utilisateur bloqué ou désactivé : 403 Forbidden.
- Whitelist des `redirect_url` par plateforme.

Codes : 302 Redirect (succès), 400 Token invalide/expiré/utilisé, 403 Utilisateur bloqué/désactivé.

9.4 Authentification Sociale (OAuth)

9.4.1 GET /auth/oauth/google

Initie le flux OAuth Google. Redirige vers l'écran de consentement.
Query params : `?platform_id=uuid&redirect_uri=https://...`
Response : 302 Redirect vers Google.

9.4.2 GET /auth/oauth/google/callback

Callback OAuth Google. Crée ou lie le compte, émet les tokens.

Sécurité des callbacks (v1.2) : les tokens ne sont **jamais** transmis dans l'URL (risque de fuite via logs, referrer, historique navigateur). Le callback pose les tokens dans des **cookies HttpOnly sécurisés** :

- `Set-Cookie: access_token=eyJ...; HttpOnly; Secure; SameSite=Lax; Path=/; Max-Age=900`
- `Set-Cookie: refresh_token=dGh...; HttpOnly; Secure; SameSite=Lax; Path=/auth/refresh; Max-Age=604800`

Response : 302 Redirect vers `redirect_uri` (sans tokens dans l'URL).
Les cookies sont envoyés automatiquement par le navigateur sur les requêtes API suivantes. Le frontend n'y accède pas en JavaScript (HttpOnly).

Règles de sécurité callback :

- Whitelist stricte des `redirect_uri` autorisées par plateforme (stockées dans `platforms`).
- Paramètre `state` anti-CSRF généré avant la redirection OAuth, validé au retour.
- Si l'email du provider existe déjà dans `users_auth` : liaison automatique au compte existant.
- Si le provider renvoie un email non vérifié : le compte est créé mais `email_verified = false`.

9.4.3 GET /auth/oauth/facebook

Identique au flux Google, avec Facebook comme provider.

9.4.4 GET /auth/oauth/facebook/callback

Callback Facebook. Même logique et même sécurité que Google callback (cookies HttpOnly).

9.5 Sécurité & Mots de passe

9.5.1 POST /auth/forgot-password

Request body :

```
{
  "email": "user@example.com"
}
```

Response 200 :

```
{
  "message": "Reset link sent if account exists"
}
```

9.5.2 POST /auth/reset-password

Request body :

```
{
  "token": "reset-token-uuid",
  "new_password": "NewSecureP@ss456"
}
```

Response 200 :

```
{
  "message": "Password reset successfully"
}
```

9.5.3 PUT /auth/change-password

Auth : Bearer token requis.
Request body :

```
{
  "current_password": "OldP@ss123",
  "new_password": "NewP@ss456"
}
```

Response 200 :

```
{
  "message": "Password changed successfully"
}
```

9.6 Authentification à Double Facteur (2FA)

9.6.1 POST /auth/2fa/enable

Auth : Bearer token requis.
Response 200 :

```
{
  "secret": "JBSWY3DPEHPK3PXP",
  "qr_code_url": "otpauth://totp/AGT:user@example.com?...",
  "message": "Scan QR code, then verify with POST /auth/2fa/confirm"
}
```

9.6.2 POST /auth/2fa/confirm

Clarification v1.1 : cet endpoint confirme l'activation du 2FA après scan du QR code. Distinct de `/auth/2fa/verify` qui sert au challenge login.

Auth : Bearer token requis.
Request body :

```
{
  "code": "482910"
}
```

Response 200 :

```
{
  "message": "2FA activated",
  "two_fa_enabled": true
}
```

9.6.3 POST /auth/2fa/verify

Vérification du code 2FA lors du login. Appelé uniquement quand `requires_2fa = true`.
Request body :

```
{
  "code": "482910",
  "temp_token": "eyJ2ZmE..."
}
```

Response 200 :

```
{
  "access_token": "eyJhbG...",
  "token_type": "Bearer",
  "expires_in": 900
}
```

Le refresh token est posé dans un cookie HttpOnly.

9.6.4 POST /auth/2fa/disable

Auth : Bearer token requis.
Request body :

```
{
  "code": "482910"
}
```

Response 200 :

```
{
  "message": "2FA disabled",
  "two_fa_enabled": false
}
```

9.7 Gestion des Sessions & Tokens

9.7.1 POST /auth/refresh

Renouvellement du JWT. Le refresh token est consommé et un nouveau est émis (rotation).

Stratégie refresh token (v1.3) : le refresh token est transmis **exclusivement via cookie HttpOnly**. Aucun body n'est requis ; le serveur lit le cookie `refresh_token` automatiquement.

Request : aucun body. Le cookie `refresh_token` est envoyé automatiquement par le navigateur.

Response 200 :

```
{
  "access_token": "eyJhbG...",
  "token_type": "Bearer",
  "expires_in": 900
}
```

Le nouveau refresh token est posé dans un cookie mis à jour :

- `Set-Cookie: refresh_token=bmV3...; HttpOnly; Secure; SameSite=Lax; Path=/auth/refresh; Max-Age=604800`

9.7.2 POST /auth/logout

Auth : Bearer token requis. Le cookie `refresh_token` est lu automatiquement.
Aucun body requis.

Response 200 :

```
{
  "message": "Logged out successfully"
}
```

Effet sur les tokens (v1.3.1) :

- La session est marquée `is_active = false`.
- Le refresh token associé est révoqué.
- Le cookie `refresh_token` est supprimé (`Max-Age=0`).
- L'access token JWT reste techniquement valide jusqu'à son expiration (15 min), mais tout appel à `verify-token` retournera `session_revoked` car la session est inactive.
- **Recommandation aux services consommateurs :** les services qui vérifient les tokens uniquement par signature locale (sans appeler `verify-token`) doivent accepter un délai de latence égal au TTL de l'access token. Les services sensibles doivent toujours appeler `verify-token` ou utiliser un cache Redis à TTL court (30s).

9.7.3 GET /auth/sessions

Auth : Bearer token requis.
Query params : `?page=1&limit=20`
Response 200 :

```
{
  "data": [
    {
      "id": "uuid",
      "platform": "agt-bot",
      "ip_address": "192.168.1.1",
      "user_agent": "Mozilla/5.0...",
      "created_at": "2026-03-19T10:00:00Z",
      "is_current": true
    }
  ],
  "page": 1,
  "limit": 20,
  "total": 3
}
```

9.7.4 DELETE /auth/sessions/{sessionId}

Auth : Bearer token requis.
Response 200 :

```
{
  "message": "Session revoked"
}
```

9.7.5 GET /auth/verify-token

*Usage : endpoint interne pour les autres services. Valide un JWT **utilisateur**.*
Headers : `Authorization: Bearer <access_token>`

Vérifications effectuées (v1.2) :

1. Signature JWT valide (RS256)
2. Token non expiré
3. Utilisateur `is_blocked = false`
4. Utilisateur `is_deactivated = false`
5. Plateforme `is_active = true`
6. Session associée `is_active = true`

Note : les vérifications 3 à 6 impliquent un appel DB. Le résultat peut être caché dans Redis (TTL court : 30s) pour limiter la charge.

Response 200 (token valide) :

```
{
  "valid": true,
  "user_id": "uuid",
  "platform_id": "agt-bot",
  "expires_at": "2026-03-19T10:15:00Z"
}
```

Response 401 (token invalide) :

```
{
  "valid": false,
  "reason": "user_blocked"
}
```

Raisons possibles : `invalid_signature`, `token_expired`, `user_blocked`, `user_deactivated`, `platform_inactive`, `session_revoked`.

Note : cet endpoint ne retourne pas de rôles. Les services consommateurs doivent interroger le Service Users pour les rôles et permissions.

9.8 Règles de Révocation des Sessions

Ajout v1.2 : règles formalisées de révocation des sessions et refresh tokens.

Événement	Effet sur les sessions / refresh tokens
`change-password`	Toutes les sessions révoquées **sauf la session courante**. Tous les refresh tokens associés invalidés.
`reset-password`	**Toutes** les sessions révoquées (l'utilisateur n'est pas connecté). Tous les refresh tokens invalidés.
`block` (admin)	**Toutes** les sessions révoquées. Tous les refresh tokens invalidés.
`unblock` (admin)	Les sessions précédentes restent révoquées. L'utilisateur doit se reconnecter.
`deactivate`	**Toutes** les sessions révoquées. Tous les refresh tokens invalidés.
`logout`	Seule la session courante est révoquée. Le refresh token associé est invalidé.
`admin_deactivate` (S2S)	**Toutes** les sessions révoquées. Tous les refresh tokens invalidés. Statut propagé à Users.
`purge` (RGPD)	Suppression physique totale du compte et de tous les artefacts Auth.

Limite de refresh tokens : un utilisateur peut avoir au maximum **5 refresh tokens actifs** simultanément. Lorsqu'un 6ème est émis, le plus ancien est automatiquement révoqué (FIFO).

Relation session / refresh token : un refresh token appartient toujours à une seule session (FK `session_id`). La révocation d'une session entraîne la révocation de son refresh token associé.

9.9 Échange de Token (Cookie → Bearer)

Ajout v1.3.2 : cet endpoint résout l'incohérence entre les callbacks (qui posent des cookies HttpOnly) et les endpoints protégés (qui attendent un header `Authorization: Bearer`).

> **Problème :** après un callback OAuth ou magic link, l'access token est dans un cookie HttpOnly. Le frontend JavaScript ne peut pas le lire pour le mettre dans le header `Authorization: Bearer`.
>
> **Solution :** un endpoint dédié qui lit le cookie et retourne l'access token dans le body JSON. Le frontend l'appelle une seule fois après la redirection.

9.9.1 POST /auth/token/exchange

Aucun body requis. Le cookie `access_token` est lu automatiquement par le serveur.

Response 200 :

```
    {
      "access_token": "eyJhbG...",
      "token_type": "Bearer",
      "expires_in": 900
    }
```

Effet : le cookie `access_token` est supprimé (`Max-Age=0`) après l'échange. À partir de ce moment, le frontend utilise le header `Authorization: Bearer` pour toutes les requêtes.

Le cookie `refresh_token` reste en place (il ne passe jamais par le frontend).

Codes : 200 OK, 401 Cookie absent ou token invalide.

Flux complet OAuth / magic link (v1.3.2) :

1. Frontend redirige vers `GET /auth/oauth/google` ou l'utilisateur clique le magic link.

2. Callback Auth pose `access_token` et `refresh_token` en cookies HttpOnly, puis redirige vers le frontend.
3. Frontend appelle `POST /auth/token/exchange` (les cookies partent automatiquement).
4. Auth retourne l'access token en JSON et supprime le cookie access_token.
5. Frontend stocke l'access token en mémoire (variable JS, pas localStorage) et l'utilise en header Bearer.
6. Pour le refresh, le navigateur envoie automatiquement le cookie refresh_token à `POST /auth/refresh`.

9.10 Profil & Audit

9.10.1 GET /auth/me

Auth : Bearer token requis.
Response 200 :

```
    {
      "id": "uuid",
      "email": "user@example.com",
      "phone": "+237600000000",
      "email_verified": true,
      "phone_verified": true,
      "two_fa_enabled": true,
      "registration_method": "email",
      "registration_platform_id": "platform-uuid",
      "is_blocked": false,
      "is_deactivated": false,
      "created_at": "2026-01-15T10:30:00Z"
    }
```

Note : aucun rôle n'est retourné. Le profil étendu et les rôles sont gérés par le Service Users.

9.10.2 GET /auth/login-history

Auth : Bearer token requis.
Query params : `?page=1&limit=20&platform=agt-bot`
Response 200 :

```
    {
      "data": [
        {
          "id": "uuid",
          "method": "email",
          "platform": "agt-bot",
          "ip_address": "192.168.1.1",
          "success": true,
          "created_at": "2026-03-19T09:45:00Z"
        }
      ],
      "page": 1,
      "limit": 20,
      "total": 142
    }
```

9.10.3 GET /auth/stats/{userId}

Auth : Bearer token requis (admin ou même utilisateur).
Response 200 :

```
{
  "user_id": "uuid",
  "total_logins": 142,
  "last_login": "2026-03-19T09:45:00Z",
  "active_sessions": 3
}
```

9.11 Gestion des Plateformes (Admin)

Ajout v1.1 : endpoints CRUD pour les plateformes, correspondant aux besoins F-PLAT-01 à F-PLAT-04. Protégés par `X-Admin-API-Key` *(voir section Administration).*

9.11.1 POST /auth/platforms

Création d'une nouvelle plateforme.
Auth : `X-Admin-API-Key: <key>`
Request body :

```
{
  "name": "AGT-Bot",
  "slug": "agt-bot",
  "allowed_auth_methods": ["email", "google", "facebook"]
}
```

Response 201 :

```
{
  "id": "uuid",
  "name": "AGT-Bot",
  "slug": "agt-bot",
  "allowed_auth_methods": ["email", "google", "facebook"],
  "is_active": true,
  "created_at": "2026-03-20T10:00:00Z"
}
```

Codes : 201 Created, 400 Validation error, 409 Slug already exists.

9.11.2 GET /auth/platforms

Liste toutes les plateformes.
Auth : `X-Admin-API-Key: <key>`
Response 200 :

```
{
  "data": [
    {
      "id": "uuid",
      "name": "AGT-Bot",
      "slug": "agt-bot",
      "allowed_auth_methods": ["email", "google"],
      "is_active": true,
      "created_at": "2026-03-20T10:00:00Z"
```

```
        }
      ]
    }
```

9.11.3 PUT /auth/platforms/{platformId}

Mise à jour d'une plateforme.
Auth : `X-Admin-API-Key: <key>`
Request body :

```
    {
      "name": "AGT-Bot v2",
      "allowed_auth_methods": ["email", "google", "facebook", "phone"]
    }
```

Response 200 : objet plateforme mis à jour.
Codes : 200 OK, 404 Not found.

9.11.4 DELETE /auth/platforms/{platformId}

Désactivation d'une plateforme (`is_active = false`). Ne supprime pas physiquement.
Auth : `X-Admin-API-Key: <key>`
Response 200 :

```
    {
      "message": "Platform deactivated",
      "is_active": false
    }
```

9.12 Administration

Modèle de sécurité admin (v1.2)

Les endpoints `/auth/admin/*` et `/auth/platforms/*` sont protégés par une **API key statique** transmise via le header `X-Admin-API-Key`. Cette clé est configurée en variable d'environnement (`ADMIN_API_KEY`).

Aucun JWT utilisateur n'est requis. Aucune dépendance vers le Service Users.

Ce mécanisme est destiné aux back-offices internes et scripts d'administration.

9.12.1 POST /auth/admin/block/{userId}

Auth : `X-Admin-API-Key: <key>`
Response 200 :

```
    {
      "message": "User blocked",
      "user_id": "uuid",
      "is_blocked": true
    }
```

Effet : l'utilisateur ne peut plus se connecter. **Toutes ses sessions actives sont révoquées** et tous ses refresh tokens sont invalidés.

9.12.2 POST /auth/admin/unblock/{userId}

Auth : `X-Admin-API-Key: <key>`
Response 200 :

```
{
  "message": "User unblocked",
  "user_id": "uuid",
  "is_blocked": false
}
```

Effet : l'utilisateur peut à nouveau se connecter. Les sessions précédentes restent révoquées ; il doit se reconnecter.

*Note v2.0 : `unblock` n'implique **pas** de réactivation. Si `is_deactivated = true`, l'utilisateur reste inactif. Aucun `status-sync` n'est envoyé à Users sur un unblock.*

9.12.3 POST /auth/account/deactivate

Soft delete côté Auth. L'utilisateur désactive son propre compte.
Auth : Bearer token requis.
Request body :

```
{
  "password": "CurrentP@ss123"
}
```

Response 200 :

```
{
  "message": "Account deactivated",
  "is_deactivated": true
}
```

Effet : le compte est marqué `is_deactivated = true`, **toutes les sessions sont révoquées**, les connexions sont refusées. Après désactivation réussie, Auth propage `status: "inactive"` au Service Users via `POST /api/v1/users/status-sync`. La suppression définitive (hard delete) est orchestrée par le Service Users.

9.12.4 POST /auth/admin/deactivate/{authUserId}

Ajout v2.0 : désactivation inter-service. Utilisé par le Service Users pour le soft delete global sans mot de passe utilisateur. Distinct de `POST /auth/account/deactivate` qui est un endpoint utilisateur.

Auth : `X-Admin-API-Key: <key>`
Response 200 :

```
{
  "message": "User deactivated",
  "user_id": "uuid",
  "is_deactivated": true
}
```

Effet : identique à `/auth/account/deactivate` : `is_deactivated = true`, toutes les sessions révoquées, tous les refresh tokens invalidés. Auth pousse ensuite `POST /api/v1/users/status-sync` avec `status: "inactive"` (voir section Contrat Auth → Users).

Codes : 200 OK, 401 API key invalide, 404 Utilisateur non trouvé.

9.12.5 DELETE /auth/admin/purge/{authUserId}

Ajout v2.0 : suppression physique RGPD. Appelé par le Service Users après avoir nettoyé le profil. Irréversible.

Auth : `X-Admin-API-Key: <key>`
Response 200 :

```
    {
      "message": "User purged",
      "user_id": "uuid",
      "purged": true
    }
```

Effet : suppression physique de toutes les données de l'utilisateur :

- `users_auth`
- `sessions`
- `refresh_tokens`
- `oauth_providers`
- `login_history`
- `verification_tokens`

Important : `authUserId` désigne `users_auth.id` (pas `users_profiles.id`).

Comportement transactionnel : l'opération de purge doit être transactionnelle. En cas d'échec partiel (ex : suppression de `sessions` réussie mais `oauth_providers` échouée), l'endpoint retourne une erreur 500 et aucune confirmation de purge ne doit être considérée comme acquise par le Service Users.

Codes : 200 OK, 401 API key invalide, 404 Utilisateur non trouvé, 500 Échec partiel de purge.

9.13 Tokens Inter-Services (Machine-to-Machine)

Communication entre Microservices

Pour qu'une plateforme (ex: AGT-Bot) puisse consommer un service interne (ex: Média), elle s'authentifie en tant que « Service » via un token S2S. Le service de destination utilise l'endpoint d'introspection pour valider ce token auprès de Auth.

9.13.1 POST /auth/s2s/token

Génère un token d'accès pour une plateforme (similaire au flux Client Credentials OAuth2).
Request body :

```
    {
      "client_id": "uuid-platform-123",
      "client_secret": "super-secret-key"
    }
```

Response 200 :

```
    {
      "access_token": "eyJhbG...",
      "token_type": "Bearer",
      "expires_in": 3600,
      "service_name": "AGT-Bot"
    }
```

9.13.2 POST /auth/s2s/introspect

*Permet à un service (ex: Média) de vérifier la validité d'un token **S2S plateforme**. Distinct de `GET /auth/verify-token` qui valide un JWT utilisateur.*
Auth : Bearer token requis (le service qui interroge doit lui-même être authentifié).
Request body :

```
{
  "token": "eyJhbG..."
}
```

Response 200 (token valide) :

```
{
  "active": true,
  "client_id": "uuid-platform-123",
  "service_name": "AGT-Bot",
  "exp": 1710846000
}
```

Response 200 (token invalide) :

```
{
  "active": false
}
```

10 Infrastructure

10.1 Composants d'infrastructure

Service	Technologie	Rôle
Base de données	PostgreSQL 15+	Stockage utilisateurs, sessions, historiques, tokens
Cache	Redis 7+	Sessions actives, OTP temporaires, rate limiting, cache verification_tokens
SMS Provider	Twilio / Vonage	Envoi des OTP SMS
Email	Notification Service	Envoi des emails transactionnels
Conteneur	Docker	Déploiement isolé et reproductible
Reverse Proxy	Nginx / Traefik	TLS termination, load balancing

10.2 Services externes à payer

Catégorie	Fournisseurs / Nature
Envoi SMS (OTP)	Twilio, Vonage, Infobip — facturation par SMS
Envoi Emails	SendGrid, AWS SES, Mailgun — facturation au volume
Hébergement & BDD	AWS, GCP, DigitalOcean — conteneurs, PostgreSQL, Redis
Sécurité / Anti-Bot	Cloudflare, reCAPTCHA — protection brute force

10.3 Variables d'Environnement

Variable	Description
`DATABASE_URL`	URL de connexion PostgreSQL
`REDIS_URL`	URL de connexion Redis
`JWT_PRIVATE_KEY`	Clé privée RS256 pour signature JWT
`JWT_PUBLIC_KEY`	Clé publique RS256 pour vérification JWT
`JWT_ACCESS_TTL`	Durée de vie access token (défaut : 900s)
`JWT_REFRESH_TTL`	Durée de vie refresh token (défaut : 604800s)
`BCRYPT_ROUNDS`	Cost factor bcrypt (défaut : 12)
`NOTIFICATION_SERVICE_URL`	URL du Service Notification
`GOOGLE_CLIENT_ID`	Client ID OAuth Google
`GOOGLE_CLIENT_SECRET`	Client Secret OAuth Google
`FACEBOOK_APP_ID`	App ID OAuth Facebook
`FACEBOOK_APP_SECRET`	App Secret OAuth Facebook
`OTP_TTL`	Durée de vie OTP en secondes (défaut : 300)
`MAGIC_LINK_TTL`	Durée de vie magic link (défaut : 600s)
`RATE_LIMIT_LOGIN`	Limite login par minute (défaut : 10)
`BRUTE_FORCE_MAX`	Tentatives avant blocage (défaut : 5)
`BRUTE_FORCE_LOCKOUT`	Durée blocage en secondes (défaut : 900)
`ADMIN_API_KEY`	Clé API statique pour les endpoints admin (v1.2)
`MAX_REFRESH_TOKENS`	Nombre max de refresh tokens actifs par user (défaut : 5)

11 Stratégie de Gestion des Tokens (v1.3)

Décision architecturale : tous les refresh tokens et les tokens d'authentification issus des callbacks (OAuth, magic link) sont transmis **exclusivement via cookies HttpOnly**. Aucun token sensible ne transite dans une URL, un body JSON ou le localStorage.

Token	Transport	Détails
Access token (JWT)	Header `Authorization: Bearer`	Retourné dans le body JSON au login email/phone, au refresh et via `POST /auth/token/exchange`. Posé temporairement en cookie HttpOnly lors des callbacks OAuth/magic link, puis échangé contre un bearer via `/auth/token/exchange`.
Refresh token	Cookie HttpOnly uniquement	`Path=/auth/refresh; Secure; SameSite=Lax; Max-Age=604800`
Temp token (2FA)	Body JSON	Retourné dans la réponse login quand `requires_2fa=true`. Courte durée (5 min).

Propriétés des cookies :

- `HttpOnly` : inaccessible en JavaScript (protection XSS).
- `Secure` : transmis uniquement sur HTTPS.

- `SameSite=Lax` : compatible avec les redirections cross-domain (OAuth, magic link) tout en protégeant contre les requêtes CSRF de type POST.
- `Path=/auth/refresh` : le cookie refresh_token n'est envoyé que sur l'endpoint de refresh.

Protection CSRF (v1.3.2) :

Les endpoints qui consomment des cookies (`/auth/refresh`, `/auth/logout`, `/auth/token/exchange`) sont exposés aux attaques CSRF. La stratégie de protection repose sur trois niveaux :

1. **SameSite=Lax** : bloque les requêtes POST cross-origin (première ligne de défense).
2. **Vérification du header `Origin` / `Referer`** : le serveur rejette les requêtes POST dont l'origine ne correspond pas aux domaines autorisés de l'écosystème AGT.
3. **Header custom obligatoire** : les endpoints cookie-based exigent un header `X-Requested-With: XMLHttpRequest`. Ce header ne peut pas être posé par un formulaire HTML classique (protection contre les CSRF simples).

Note : un token CSRF classique (double submit cookie) n'est pas nécessaire grâce à la combinaison SameSite=Lax + vérification Origin + header custom. Si l'architecture évolue vers du cross-domain complet, cette stratégie devra être réévaluée.

12 Rate Limiting

Endpoint	Fenêtre	Limite	Stratégie
/auth/login	1 min	10 req	Par IP + email
/auth/login/phone	1 min	3 req	Par IP + phone
/auth/verify-otp	1 min	5 req	Par IP + phone
/auth/forgot-password	15 min	3 req	Par IP
/auth/register	1 heure	5 req	Par IP
/auth/2fa/verify	1 min	5 req	Par user
Global	1 min	100 req	Par IP (tous endpoints)

Implémentation via Redis avec algorithme sliding window.

13 Contrat d'Intégration Auth → Notification

Ajout v1.3.1 : mini-contrat technique pour formaliser la communication entre Auth et le Service Notification.

13.1 Mode de communication

Paramètre	Valeur
Transport	HTTP POST (synchrone pour le MVP, migration vers message queue recommandée en production)
Format	JSON
Auth	Token S2S (Auth s'authentifie auprès de Notification via `POST /auth/s2s/token`)
Timeout	5 secondes max
Retry	3 tentatives avec backoff exponentiel (1s, 2s, 4s)

13.2 Payload standard

```
{
  "type": "email_verification",
  "recipient": {
    "email": "user@example.com",
    "phone": null
  },
  "template": "auth_verify_email",
  "data": {
    "verification_url": "https://auth.agt.com/...",
    "expires_in_minutes": 60,
    "platform_name": "AGT-Bot"
  },
  "priority": "high",
  "idempotency_key": "uuid-unique"
}
```

13.3 Types de notifications émis par Auth

Type	Canal	Déclencheur	Templ
`email_verification`	Email	POST /auth/register (method=email)	auth_v
`phone_otp`	SMS	POST /auth/register (method=phone), POST /auth/login/phone	auth_o
`password_reset`	Email	POST /auth/forgot-password	auth_r
`magic_link`	Email	POST /auth/login/magic-link	auth_n
`login_alert`	Email	Login depuis nouvelle IP/device (évolution future)	auth_lo

13.4 Comportement en cas d'échec

Scénario	Comportement Auth
Notification indisponible (timeout/5xx)	Auth crée l'utilisateur/le token quand même, mais retourne un warning : `"notification_sent": false`. L'utilisateur peut redemander l'envoi.
Notification rejette (4xx)	Auth logge l'erreur. L'opération Auth (register, reset) reste validée.
Retry épuisés	Auth logge un événement critique. Un cron job peut retenter les notifications en échec.

*Règle fondamentale : l'échec de Notification ne doit **jamais** bloquer une opération Auth. L'utilisateur doit toujours pouvoir redemander l'envoi.*

14 Contrat d'Intégration Auth → Users

Ajout v2.0 : contrat technique pour formaliser la communication entre Auth et le Service Users.

14.1 Provisioning

Après une inscription réussie (`POST /auth/register` ou callback OAuth), Auth appelle le Service Users pour créer le profil associé :

```
POST /api/v1/users
{
  "auth_user_id": "users_auth.id",
  "first_name": "",
```

```
      "last_name": "",
      "email": "user@example.com",
      "phone": "+237600000000"
    }
```

14.2 Synchronisation de statut

Lorsque `is_deactivated` change dans Auth, Auth pousse la mise à jour vers Users :

```
    POST /api/v1/users/status-sync
    {
      "auth_user_id": "uuid",
      "status": "inactive"
    }
```

Règles :

- Auth pousse `inactive` quand `is_deactivated = true`.
- Auth ne pousse **jamais** `deleted` ni `deletion_in_progress` (états internes à Users).
- `is_blocked` n'est **pas** propagé à Users : c'est une mesure de sécurité Auth, pas un changement de statut profil.
- Auth ne pousse actuellement que `inactive`. **Aucun flux de réactivation n'est défini en v2.1.** Si ce besoin émerge, un endpoint `POST /auth/admin/reactivate/{authUserId}` devra être ajouté.

14.3 Synchronisation email / téléphone

Lorsque l'email ou le téléphone change dans Auth (ex : ajout d'un téléphone après inscription email), Auth pousse la mise à jour vers Users :

```
    POST /api/v1/users/sync
    {
      "auth_user_id": "uuid",
      "email": "new@example.com",
      "phone": "+237600000000"
    }
```

Règles :

- Déclenché quand `email` ou `phone` change dans `users_auth`.
- Users ne modifie **jamais** seul ces champs : Auth est la source de vérité pour les identifiants d'authentification.
- Seuls les champs modifiés sont envoyés.

14.4 Mapping de statuts Auth ↔ Users

État Auth	Projection Users	Commentaire
`is_deactivated = false`	Aucune projection automatique	État par défaut. Aucun push vers Users (pas de flux de réactivation en v2.1)
`is_deactivated = true`	`inactive`	Push via `status-sync`
`is_blocked = true`	Aucune projection	Mesure de sécurité Auth uniquement
Purge RGPD	`deleted`	Orchestré par Users, pas par Auth

15 Recommandations & Évolutions

- **Gestion des comptes liés :** un utilisateur inscrit par email peut lier son compte Google/Facebook via la table `oauth_providers`. Lors d'un login OAuth, le service vérifie d'abord si l'email existe dans `users_auth`. Si oui, il lie automatiquement le provider au compte existant (après confirmation utilisateur).
- **Nettoyage des sessions :** un cron job quotidien doit purger les sessions expirées, les refresh tokens révoqués et les verification_tokens utilisés ou expirés.
- **Monitoring :** exposer des métriques Prometheus (nombre de logins, erreurs 401/403, latence P95, sessions actives).
- **Versioning API :** préfixe `/api/v1`. En cas de breaking changes, publier une `/api/v2` avec période de cohabitation.
- **Migration future :** si le besoin de SSO émerge, envisager une couche OpenID Connect par-dessus le système JWT existant.

Annexes

Glossaire

Terme	Définition
JWT	JSON Web Token — standard pour transmettre des informations signées
Access Token	Jeton à durée courte (15 min) prouvant l'authentification
Refresh Token	Jeton à durée longue (7 jours) pour renouveler l'access token
OAuth	Protocole d'autorisation via compte tiers (Google, Facebook)
2FA / TOTP	Authentification à double facteur via code temporaire
OTP	Mot de passe à usage unique, envoyé par SMS ou email
Magic Link	Lien d'authentification unique envoyé par email
Rate Limiting	Limitation du nombre de requêtes par fenêtre de temps
Stateless	Architecture où le serveur ne conserve pas d'état en mémoire locale
S2S	Service-to-Service — communication entre microservices
Introspect	Vérification de validité d'un token S2S par un service tiers
Soft Delete	Désactivation logique sans suppression physique

Code PlantUML — Diagramme de Classes

Le code PlantUML ci-dessous correspond au diagramme de classes complet du service Auth v2.1. Il peut être rendu sur `plantuml.com` ou tout outil compatible.

```
@startuml Auth Service - Class Diagram v2.1

skinparam classAttributeIconSize 0
skinparam linetype ortho

entity "platforms" as platforms {
  * id : UUID <<PK>>
  --
  * name : VARCHAR(50) <<UQ>>
  * slug : VARCHAR(50) <<UQ>>
  * allowed_auth_methods : JSONB
  * allowed_redirect_urls : JSONB
  * client_secret_hash : VARCHAR(255)
  is_active : BOOLEAN = true
  * created_at : TIMESTAMPTZ
  * updated_at : TIMESTAMPTZ
```

```
    }

    entity "users_auth" as users {
      * id : UUID <<PK>>
      --
      email : VARCHAR(255) <<UQ>>
      phone : VARCHAR(20) <<UQ>>
      password_hash : VARCHAR(255)
      email_verified : BOOLEAN = false
      phone_verified : BOOLEAN = false
      two_fa_enabled : BOOLEAN = false
      two_fa_secret : VARCHAR(255)
      * registration_method : VARCHAR(20)
      * registration_platform_id : UUID <<FK>>
      failed_login_attempts : INT = 0
      locked_until : TIMESTAMPTZ
      is_blocked : BOOLEAN = false
      is_deactivated : BOOLEAN = false
      * created_at : TIMESTAMPTZ
      * updated_at : TIMESTAMPTZ
    }

    entity "sessions" as sessions {
      * id : UUID <<PK>>
      --
      * user_id : UUID <<FK>>
      * platform_id : UUID <<FK>>
      * ip_address : VARCHAR(45)
      user_agent : TEXT
      is_active : BOOLEAN = true
      * created_at : TIMESTAMPTZ
      * expires_at : TIMESTAMPTZ
    }

    entity "refresh_tokens" as rtokens {
      * id : UUID <<PK>>
      --
      * user_id : UUID <<FK>>
      * session_id : UUID <<FK>>
      * token_hash : VARCHAR(255) <<UQ>>
      is_revoked : BOOLEAN = false
      * created_at : TIMESTAMPTZ
      * expires_at : TIMESTAMPTZ
    }

    entity "oauth_providers" as oauth {
      * id : UUID <<PK>>
      --
      * user_id : UUID <<FK>>
      * provider : VARCHAR(20)
      * provider_user_id : VARCHAR(255)
      email : VARCHAR(255)
      * created_at : TIMESTAMPTZ
      ..
      <<UQ>> (user_id, provider)
      <<UQ>> (provider, provider_user_id)
    }

    entity "login_history" as loghist {
      * id : UUID <<PK>>
      --
      * user_id : UUID <<FK>>
      * platform_id : UUID <<FK>>
      * method : VARCHAR(20)
      * ip_address : VARCHAR(45)
      user_agent : TEXT
      * success : BOOLEAN
      failure_reason : VARCHAR(100)
      * created_at : TIMESTAMPTZ
    }

    entity "verification_tokens" as vtokens {
      * id : UUID <<PK>>
```

```
      --
      * user_id : UUID <<FK>>
      * token_hash : VARCHAR(255) <<UQ>>
      * type : VARCHAR(30)
      payload : JSONB
      * expires_at : TIMESTAMPTZ
      used_at : TIMESTAMPTZ
      * created_at : TIMESTAMPTZ
    }

    platforms ||--o{ sessions       : "1..* sessions"
    platforms ||--o{ loghist        : "1..* login_history"
    platforms ||--o{ users          : "1..* registered on"

    users ||--o{ sessions          : "1..* sessions"
    users ||--o{ rtokens           : "1..* refresh_tokens"
    users ||--o{ oauth             : "1..* oauth_providers"
    users ||--o{ loghist           : "1..* login_history"
    users ||--o{ vtokens           : "1..* verification_tokens"

    sessions ||--o| rtokens        : "1..0..1 refresh_token"

    @enduml
```